Introduction to Algorithms

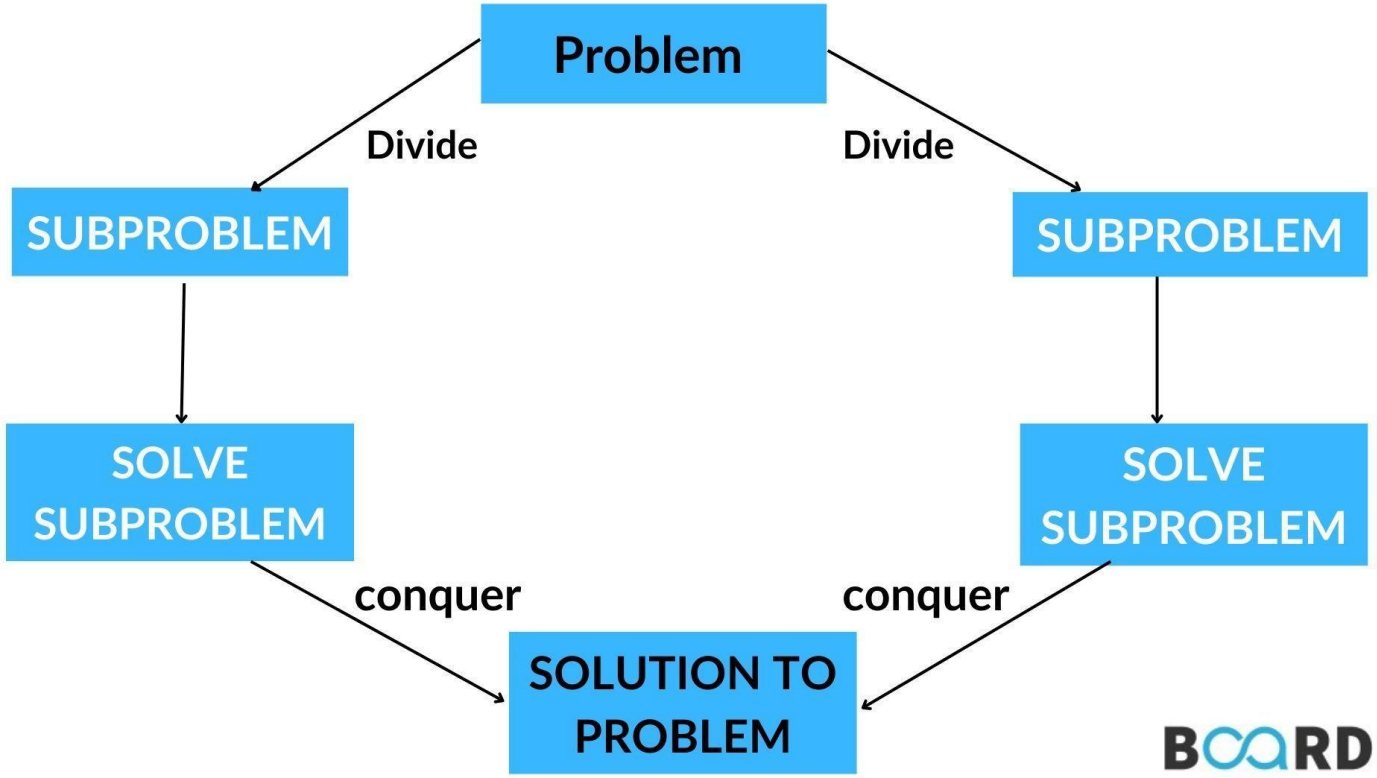
Levon Prazyan

Course Overview

- **Introduction**
  - **1 lecture**
- **Analyzing Algorithms**
  - **2 lectures**
- **Recurrences**
  - **2 lectures**
- **Probabilistic Analysis and Randomized Algorithms**
  - **2 lecture**
- **Sorting and Searching**
  - **4 lectures**
- **Heaps**
  - **1 lecture**
- **Arrays, Lists**
  - **1 lectures**
- **Stack, Queue, Double Ended Queue**
  - **2 lectures**

Devide and Counquer

- Divide
- Conquer
- Combine



The maximum-subarray problem

- Given an array A. Need to find non-empty contiguous subarray of A with maximum summary of subarray elements.

{8, -10, **2, 0, -1, 3, -1, 7**, -3}

1. brute-force
   Try every $\{i, j\}$ pairs and calculate elements between i and j.
   Take maximum of calculated values. We will use $\Theta(n^2)$ time.
2. divide and conquer

The maximum-subarray problem: Divide and conquer (1)

Suppose the input is $A[low\,..\,high]$ array.

$mid = (low + high) / 2$ – the midpoint.

$A[low\,..\,mid]$, $A[mid + 1\,..\,high]$ – subarrays

Any $A[i, j]$ congested subarray must lie in exactly one of the followings:

1. entirely in subarray – $A[low\,..\,mid]$; $low \leq i \leq j \leq mid$
2. entirely in subarray – $A[mid + 1\,..\,high]$; $\text{mid} < i \leq j \leq high$
3. crossing the midpoint – $low \leq i \leq mid < j \leq high$

The maximum-subarray problem: Divide and conquer (2)

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 1  left-sum = −∞
 2  sum = 0
 3  for i = mid downto low
 4       sum = sum + A[i]
 5       if sum > left-sum
 6            left-sum = sum
 7            max-left = i
 8  right-sum = −∞
 9  sum = 0
10  for j = mid + 1 to high
11       sum = sum + A[j]
12       if sum > right-sum
13            right-sum = sum
14            max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1  if high == low
 2       return (low, high, A[low])          // base case: only one
 3  else mid = ⌊(low + high)/2⌋
 4       (left-low, left-high, left-sum) =
              FIND-MAXIMUM-SUBARRAY(A, low, mid)
 5       (right-low, right-high, right-sum) =
              FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
 6       (cross-low, cross-high, cross-sum) =
              FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 7       if left-sum ≥ right-sum and left-sum ≥ cross-sum
 8            return (left-low, left-high, left-sum)
 9       elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
10            return (right-low, right-high, right-sum)
11       else return (cross-low, cross-high, cross-sum)
```

The maximum-subarray problem: Divide and conquer: Analysis (1)

```
FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
1   left-sum = −∞
2   sum = 0
3   for i = mid downto low
4       sum = sum + A[i]
5       if sum > left-sum
6           left-sum = sum
7           max-left = i
8   right-sum = −∞
9   sum = 0
10  for j = mid + 1 to high
11      sum = sum + A[j]
12      if sum > right-sum
13          right-sum = sum
14          max-right = j
15  return (max-left, max-right, left-sum + right-sum)
```

Operations

1. c_1
2. c_2
3. $(mid - low + 2) * c_3$
4. $(mid - low + 1) * c_4$
5. $(mid - low + 1) * c_5$
6. $(mid - low + 1) * c_6$
7. $(mid - low + 1) * c_5$
8. c_8
9. c_9
10. $(high - mid + 1) * c_{10}$
11. $(high - mid) * c_{11}$
12. $(high - mid) * c_{12}$
13. $(high - mid) * c_{13}$
14. $(high - mid) * c_{14}$
15. c_{15}

The maximum-subarray problem: Divide and conquer: Analysis (2)

- $c_1 = c_2 = c_8 = c_9;$
- $c_3 = c_{10};$
- $c_4 = c_{11};$
- $c_5 = c_{12};$
- $c_6 = c_{13};$
- $c_7 = c_{14}$
- $T(n) = c + (mid - low + 1 + high - mid) * (c_3 + c_4 + c_5 + c_6 + c_7)$
- $T(n) = (high - low + 1) * a + b;$ $\quad T(n) = a * n + b_1$
- $\boldsymbol{T(n) = \Theta(n)}$

The maximum-subarray problem: Divide and conquer: Analysis (1)

```
FIND-MAXIMUM-SUBARRAY(A, low, high)
 1  if high == low
 2       return (low, high, A[low])          // base case: only one
 3  else mid = ⌊(low + high)/2⌋
 4       (left-low, left-high, left-sum) =
              FIND-MAXIMUM-SUBARRAY(A, low, mid)
 5       (right-low, right-high, right-sum) =
              FIND-MAXIMUM-SUBARRAY(A, mid + 1, high)
 6       (cross-low, cross-high, cross-sum) =
              FIND-MAX-CROSSING-SUBARRAY(A, low, mid, high)
 7       if left-sum ≥ right-sum and left-sum ≥ cross-sum
 8            return (left-low, left-high, left-sum)
 9       elseif right-sum ≥ left-sum and right-sum ≥ cross-sum
10            return (right-low, right-high, right-sum)
11       else return (cross-low, cross-high, cross-sum)
```

Operations

1. c_1
2. c_2
3. c_3
4. $T(\frac{n}{2})$
5. $T(\frac{n}{2})$
6. $T(n)$
7. c_7
8. c_8
9. c_9
10. c_{10}
11. c_{11}

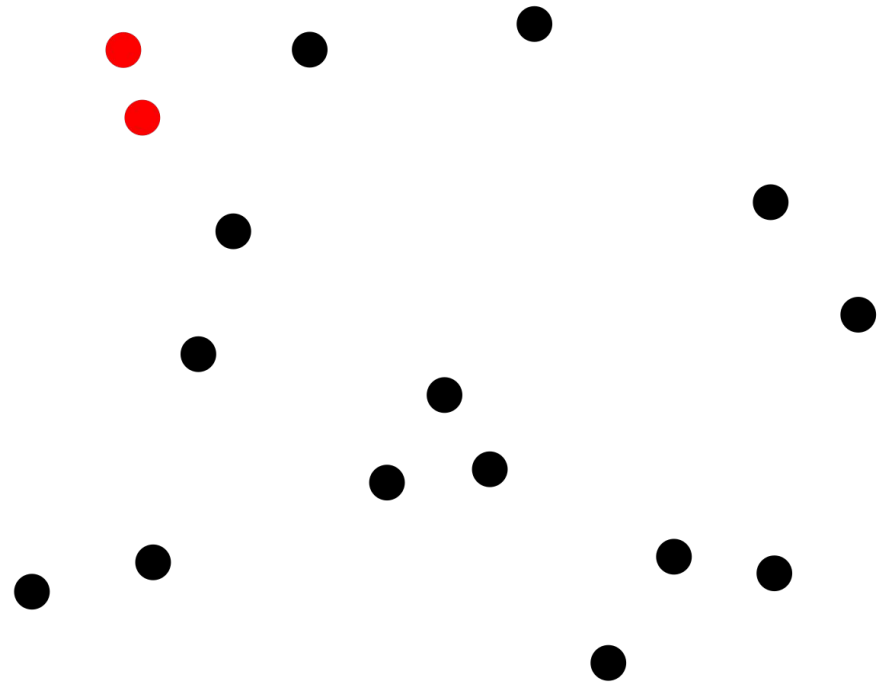
$$T(n) = 2 * T(n/2) + \Theta(n)$$

$$T(n) = \Theta(n * logn)$$

Closest Pair of Points (1)

Consider the problem of finding the closest pair of points in a set Q of $n \geq 2$ points.

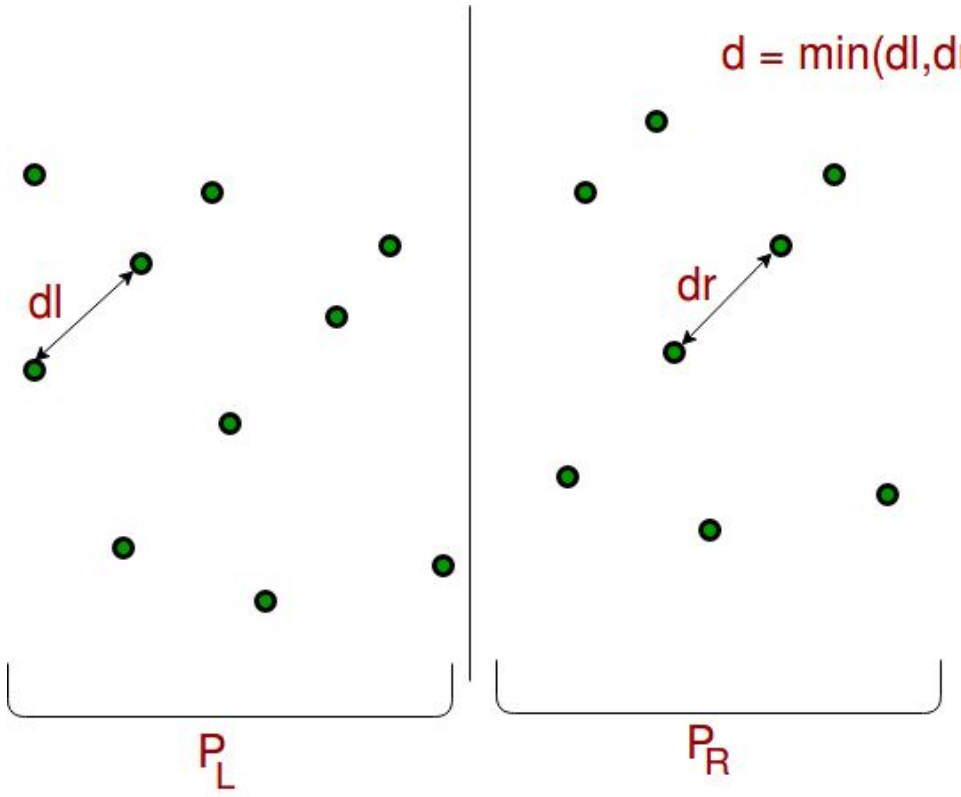
- $p_1 = (x_1, y_1)$,
- $p_2 = (x_2, y_2)$,
- $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$



Closest Pair of Points: Divide-and-Conquer. (2)

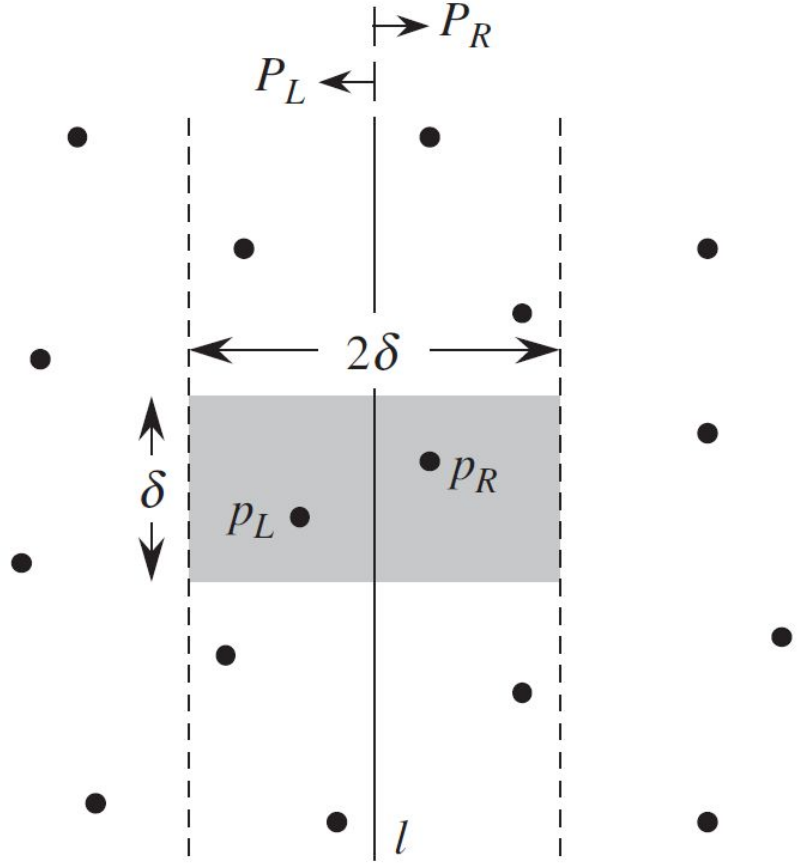
Recursive Algorithm takes a point set P.

- Let X contain all the points sorted by x-coordinate.
- Let Y contain all the points sorted by y-coordinate.
- Recursively divide the point set P into two sets P_L and P_R such that : $|P_L| = \lceil |P|/2 \rceil$, $|P_R| = \lfloor |P|/2 \rfloor$,
- Find the minimum distance be the d_l and d_r from the P_L and P_R appropriately and $d = \min(d_l, d_r)$
- The closest pair is either the pair with distance d or it is a distance between a pair of points with one point in P_L and P_R and the other in P_R

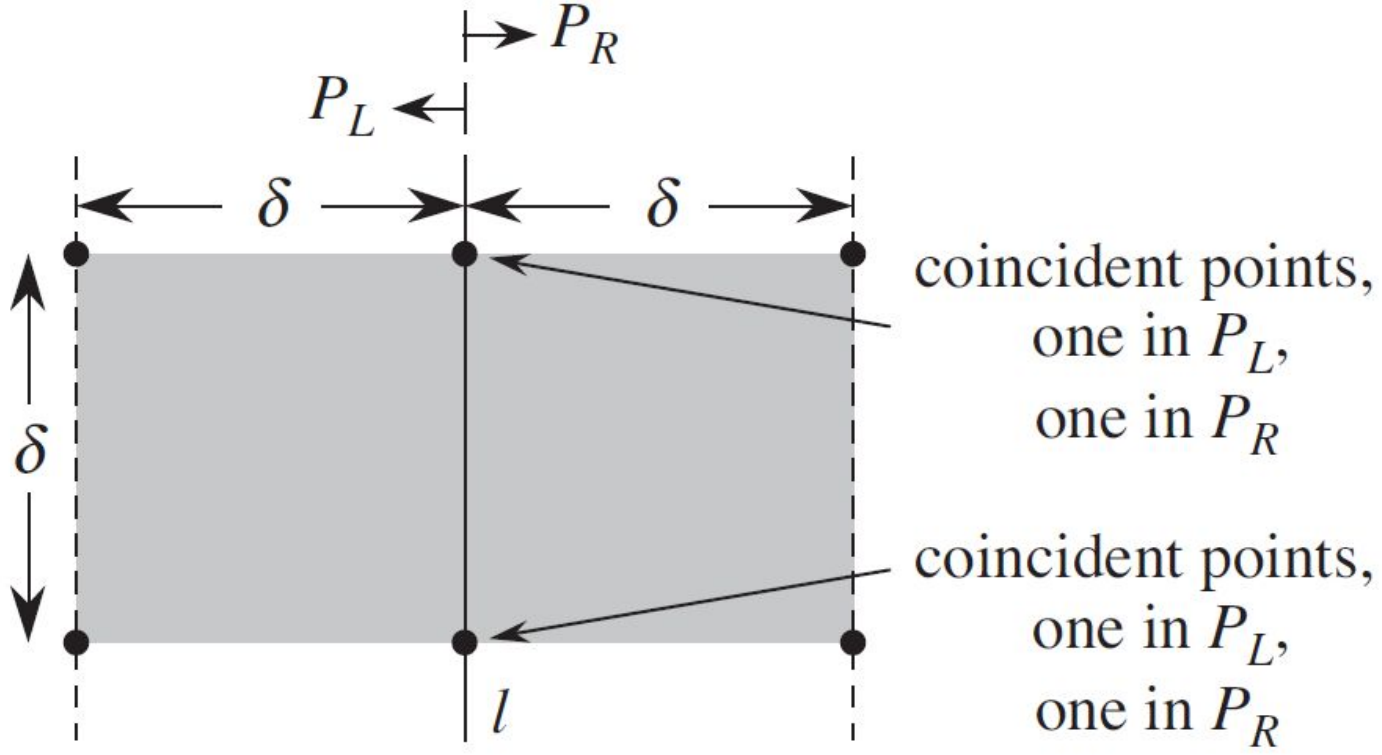


Closest Pair of Points: Divide-and-Conquer. (3)

- If a pair of points has distance less than d, both must reside in the $2d$-wide vertical strip centered at line l.
- Create an array Y', which is the array Y with all points not in the $2d$-wide vertical strip removed.
- For each point p in the array Y', try to find points in Y' that are within d units of p: d'. At most 7 points need to check for every p
- Return the smallest $d' < d$ found during the previous steps.



Closest Pair of Points: Divide-and-Conquer. (4)



- Maximum 4 points that are pairwise at least d units apart can all reside within a $d \times d$ square.
- The $2d \times d$ square can contain 8 points.

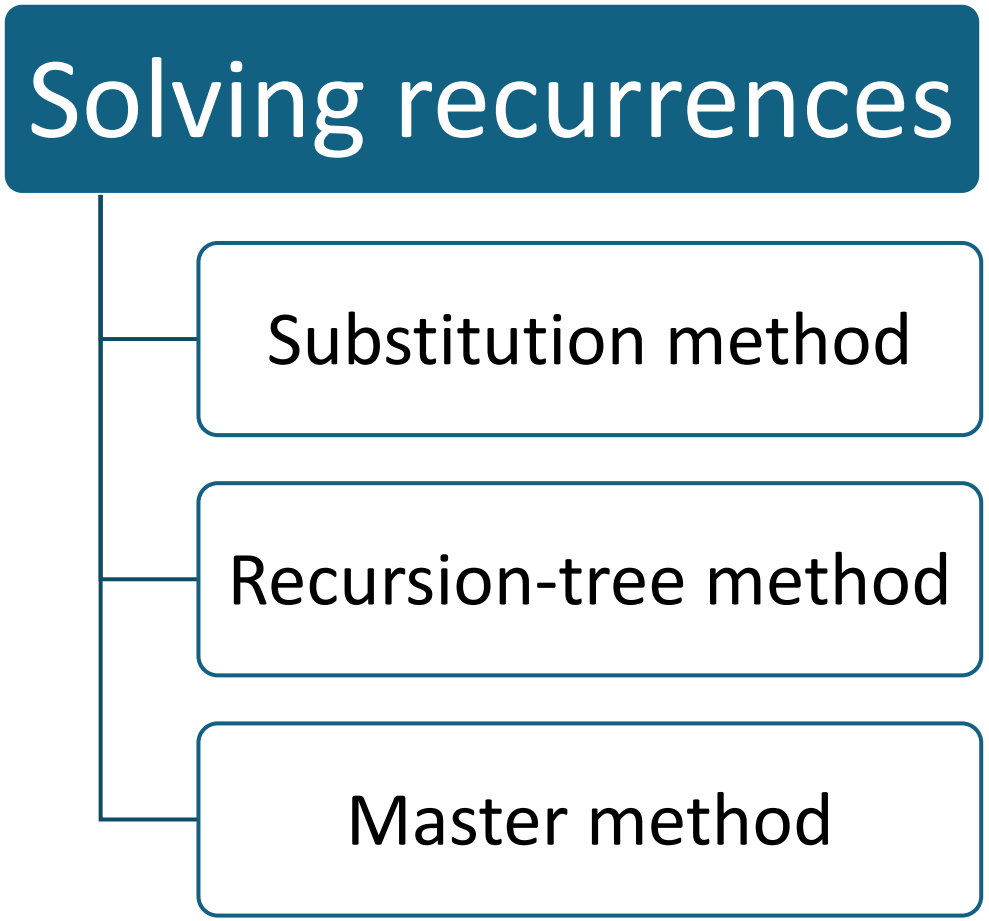
Closest Pair of Points: Divide-and-Conquer. (5)

```
01 int closestPairPoints(Points X, Points Y)
02 {
03    int n = X.size();
04    if (n == 2) return dist(X[0], X[1]);
05    if (n == 3) return min(dist(X[0], X[1]), dist(X[1], X[2]), dist(X[0], X[2]));
06
07    mid = n / 2;
08    {Yl, Yr} = divide(Y, X);
09    d = min(closestPairPoints(X[0..mid], Yl),
10            closestPairPoints(X[mid+1..n], Yr));
11
12    StripeY = getStripElements(Y, X[mid].x, d);
13    for i = 1 to StripeY.length
14       for j = 1 to 7
15          if (i + j <= StripeY.length)
16             d = min(d. dist(StripeY[i], StripeY[i+j]))
17    return d;
18 }
```

Recursion

- Recursion is a concept where a procedure calls itself in order to solve a problem.
  A recursive algorithm typically follows these steps:
  - Base Case
  - Recursive Case

Solving Recurrences



The Substitution Method

Guess the solution

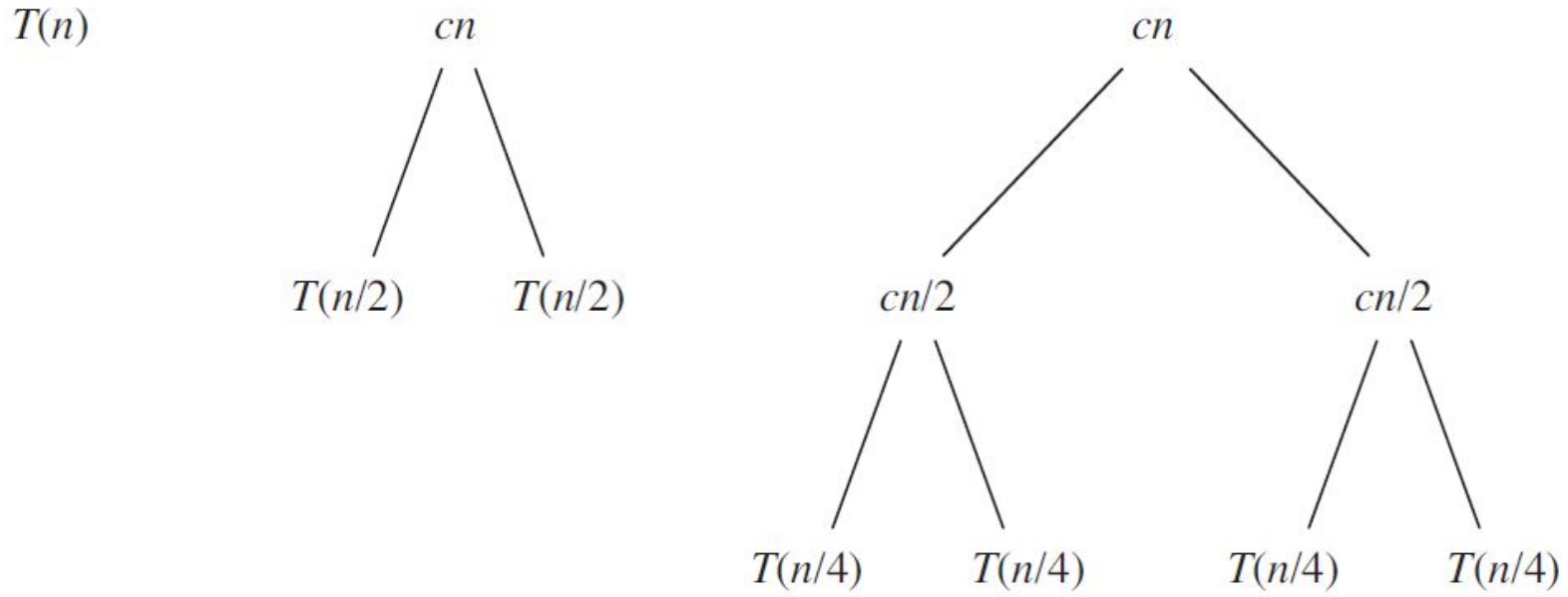
Proof by induction

The Substitution Method: Example

- Given the formula: $T(n) = 2 * T(n/2) + n$
- The guess is $T(n) = \Theta(n * logn)$
- Assumption: the guess is correct for all $m < n$, where $m = n/2$
- $c_1 * n/2 * \log\frac{n}{2} \leq \boldsymbol{T(n/2)} \leq c_2 * n/2 * \log\frac{n}{2}$
- $2 * (c_1 * n/2 * \log\frac{n}{2}) + n \leq \boldsymbol{2 * T\left(\frac{n}{2}\right) + n} \leq 2 * (c_2 * n/2 * \log\frac{n}{2}) + n$
- $c_1 * n * \log n - c_1 * n * \log 2 + n \leq \boldsymbol{T(n)} \leq c_2 * n * \log n - c_2 * n * \log 2 + n$
- $c_1 * n * \log n - c_1 * n + n \leq \boldsymbol{T(n)} \leq c_2 * n * \log n - c_2 * n + n$
- $c_1 * n * \log n - c_1 * n + n \leq \boldsymbol{T(n)} \leq c_2 * n * \log n - c_2 * n + n$
- $c_1 * n * \log n \leq \boldsymbol{T(n)} \leq c_2 * n * \log n$; when $c_1 \leq 1 \leq c_2$

The Recursive-tree Method: Example 1 (1)

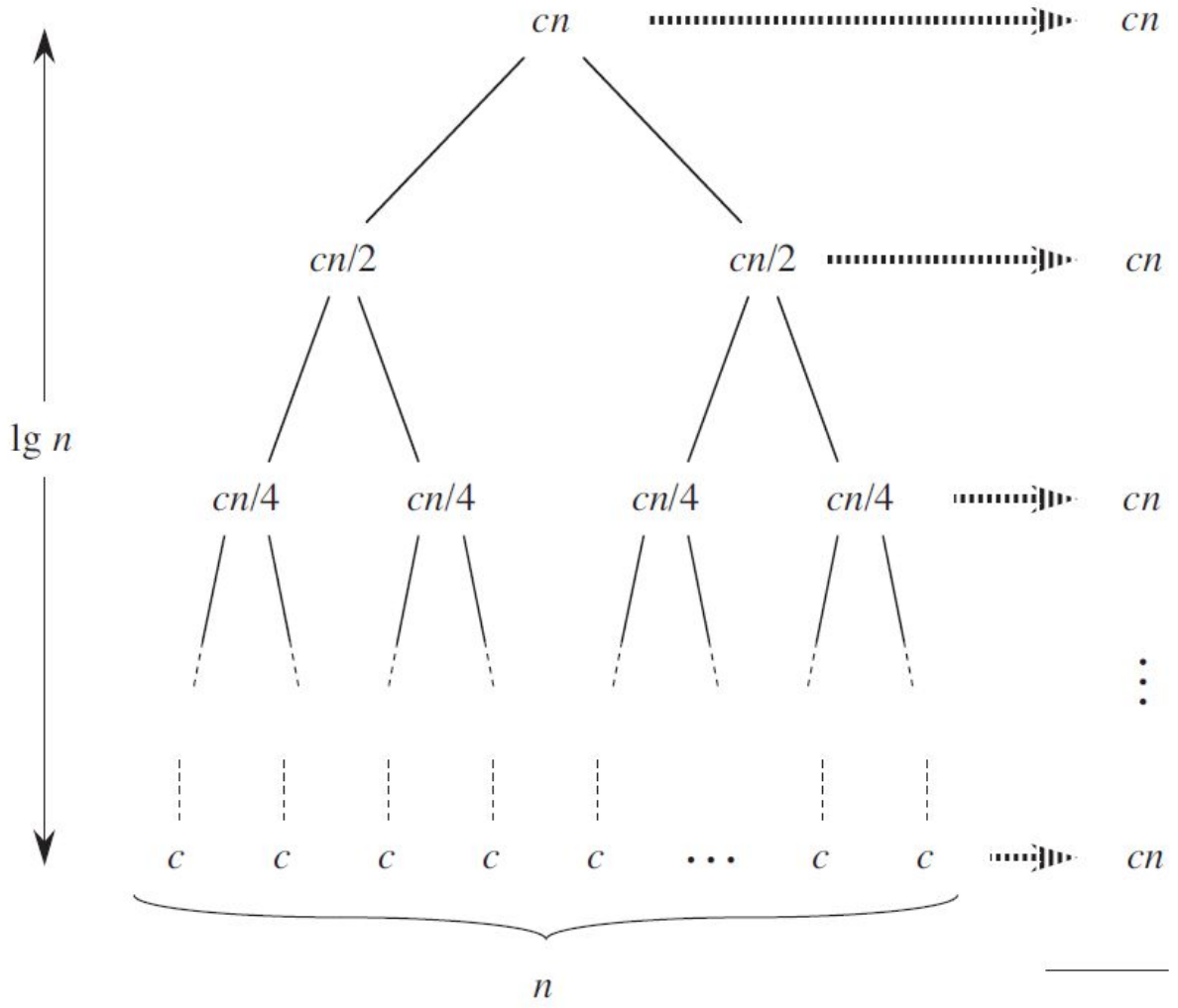
- $$T(n) = \begin{cases} cn; & n = 1 \\ 2T(n/2) + cn; & n > 1 \end{cases}$$



The Recursive-tree Method: Example 1 (2)

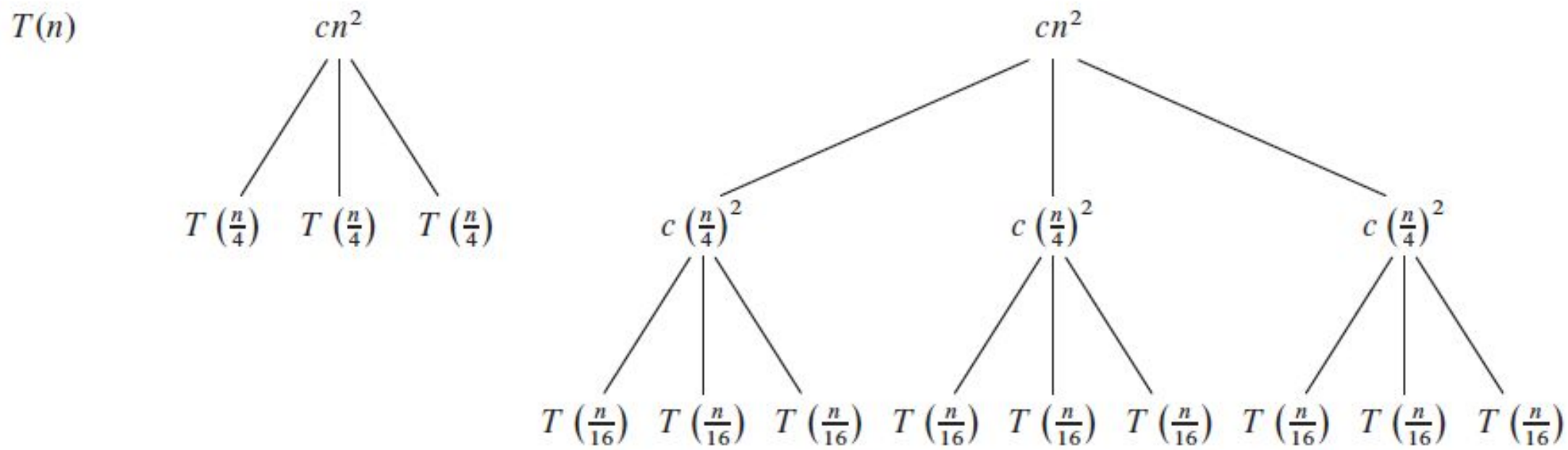
- $T(n) = cn \log n + cn$

$$T(n) = \Theta(n \cdot \log n)$$

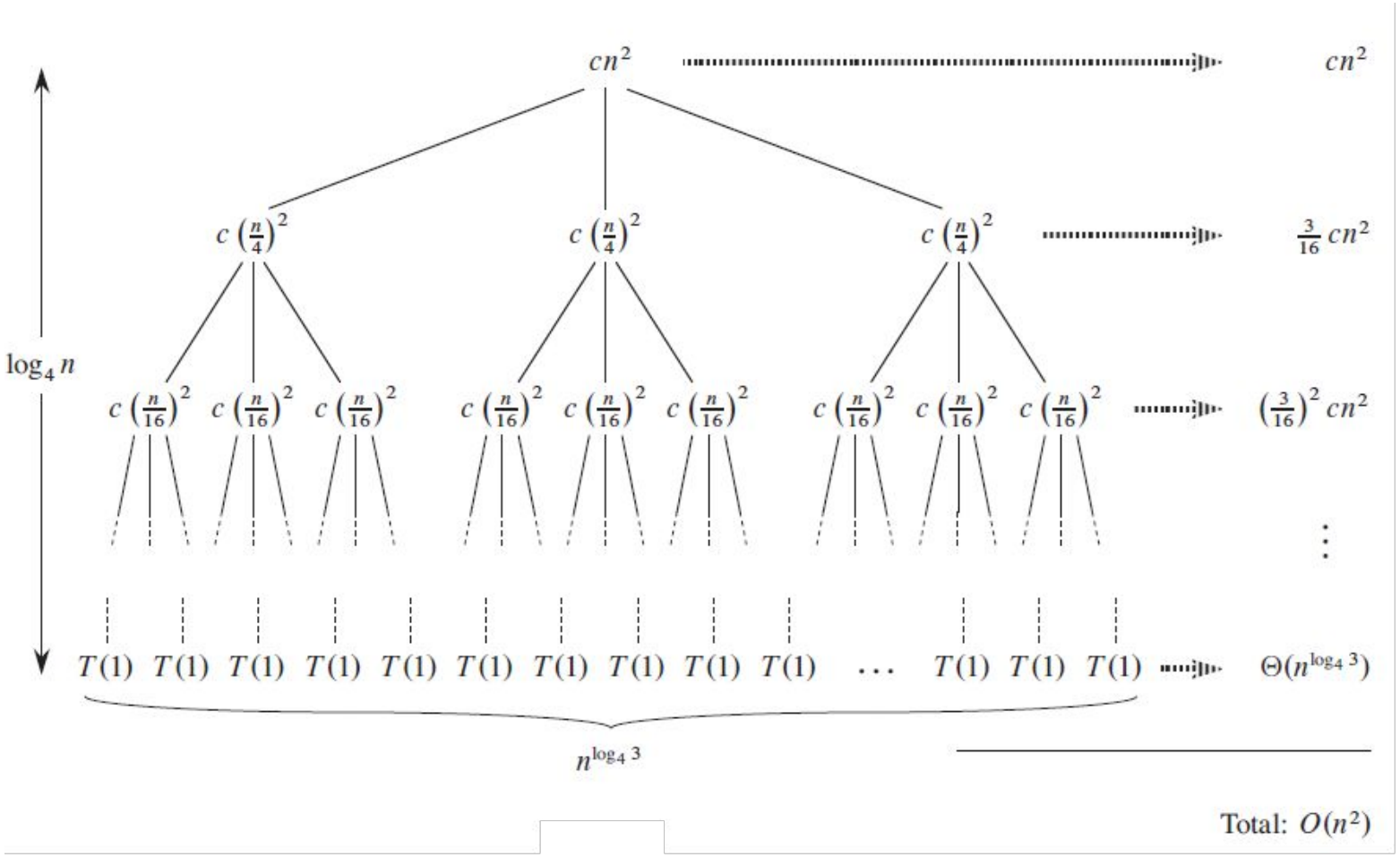


The Recursive-tree Method: Example 2 (1)

- $$T(n) = \begin{cases} c; & n = 1 \\ 3T(n/4) + cn^2; & n > 1 \end{cases}$$



The Recursive-tree Method: Example 2 (2)



- $$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$T(n) < \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$T(n) < \frac{16}{13} cn^2 + \Theta\left(n^{\log_4 3}\right)$$

$$\boldsymbol{T(n) = O(n^2)}$$

Exercises (1)

1. Show that the solution of $T(n) = T(n-1) + n/2$ is $O(n^2)$
2. Show that the solution of $T(n) = T(n/2) + 1$ is $O(\log n)$
3. Show that the solution of $T(n) = 2T(n/2) + n/2$ is $\Omega(n \log n)$
4. Show that the solution of $T(n) = 2T(\frac{n}{2} + 7) + 8n$ is $\mathrm{O}(n \log n)$
5. Solve the recurrence $T(n) = 3T(\sqrt{n}) + \log n$
6. Solve the recurrence $T(n) = 4T(n/2) + n^2$
7. Solve the recurrence $T(n) = 4T(n/3) + n$
8. Solve the recurrence $T(n) = 3T(n/2) + n$

Exercises (2)

9. Solve the recurrence $T(n) = 3T(\frac{n}{2}) + n^2$
10. Solve the recurrence $T(n) = 4T(\frac{n}{2} + 2) + n$
11. Solve the recurrence $T(n) = 2T(n-1) + 1$
12. Solve the recurrence $T(n) = T(n-1) + T(\frac{n}{2}) + n$

Master Method

Let: $\boldsymbol{T(n) = aT(n/b) + f(n)}; a \geq 1, b > 1, f(n) > 0, n \in N$

1. if $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for some constant $\varepsilon > 0$, then
$$T(n) = \theta\left(n^{\log_b a}\right)$$
2. if $f(n) = \theta\left(n^{\log_b a}\right)$, then
$$T(n) = \theta\left(n^{\log_b a} \log n\right)$$
3. if $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ for some constant $\varepsilon > 0$, and if $af\left(\frac{n}{b}\right) \leq cf(n)$ for some $c < 1$, then
$$T(n) = \theta(f(n))$$

Master Method: Proof (1)

Analyze the for

Extend the analysis to all positive integers.

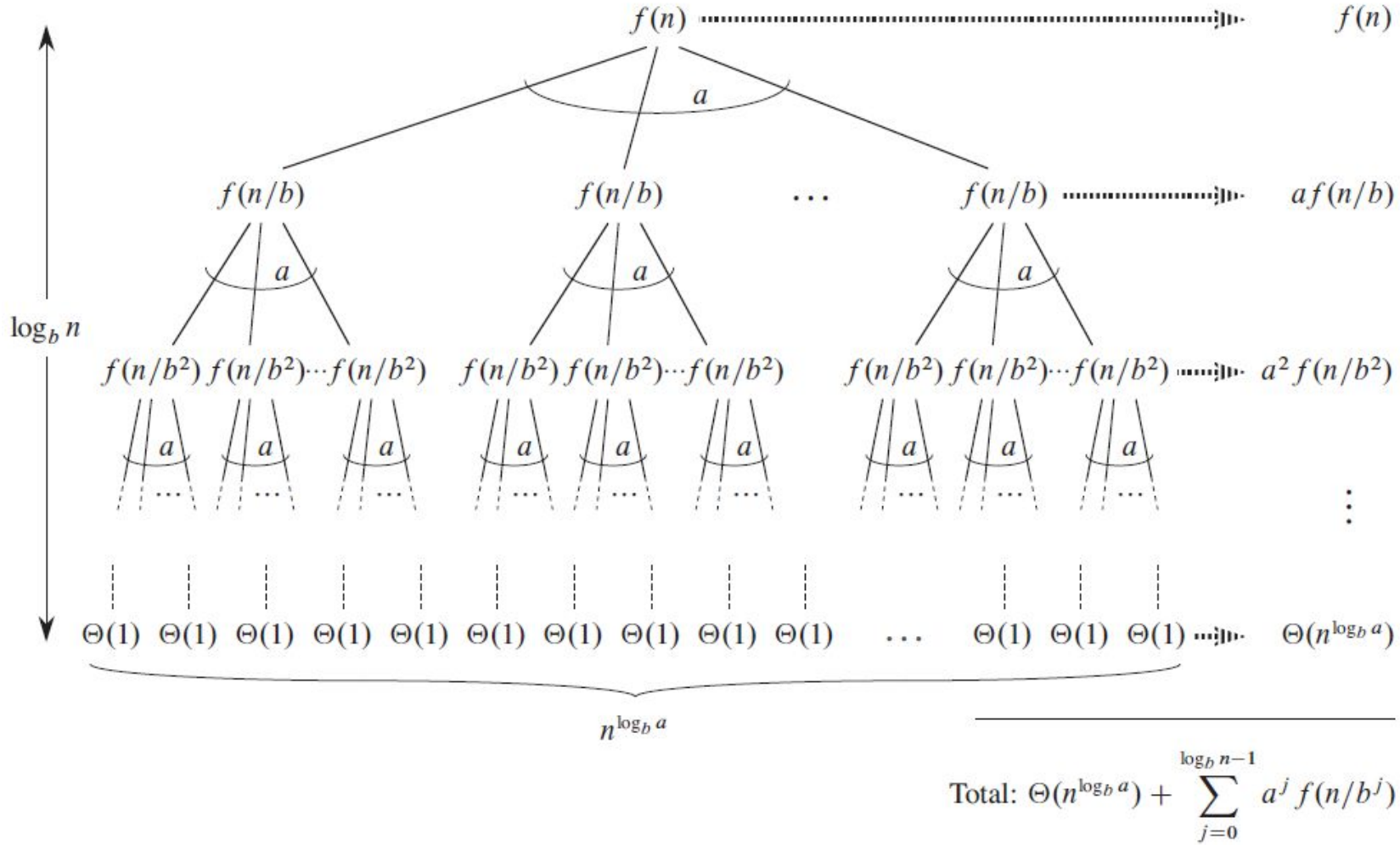
Master Method: Proof (2)

- Lemma 1:

Given $T(n) = \begin{cases} \theta(1) & if\ n = 1 \\ aT(n/b) + f(n) & if\ n = b^i \end{cases}$ $a \geq 1, b > 1, f(n) > 0$

Show that $T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left({}^{n}/_{b^j}\right)$

Master Method: Proof (3)



Master Method: Proof (4)

- Lemma 2:

Given $g(n) = \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$ where $a \geq 1,\ b > 1, f(n) > 0$

Show that:

1. if $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for $\varepsilon > 0$, then $g(n) = O\left(n^{\log_b a}\right)$
2. if $f(n) = \theta\left(n^{\log_b a}\right)$, then $g(n) = \theta\left(n^{\log_b a} \log n\right)$
3. if $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ for $\varepsilon > 0$, and if $af(n/b) \leq cf(n)$ for some $c < 1$, then $g(n) = \theta(f(n))$

Master Method: Proof (5)

- Lemma 2: Proof 1: $f(n) = O\left(n^{\log_b a-\varepsilon}\right)$ for $\varepsilon > 0$;

$$g(n) = O\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a-\varepsilon}\right) = O\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a-\varepsilon}\right)$$

$$g(n) = O\left(n^{\log_b a-\varepsilon} \sum_{j=0}^{\log_b n-1} a^j \left(\frac{b^\varepsilon}{b^{\log_b a}}\right)^j\right) = O\left(n^{\log_b a-\varepsilon} \sum_{j=0}^{\log_b n-1} \left(\frac{ab^\varepsilon}{b^{\log_b a}}\right)^j\right)$$

$$g(n) = O\left(n^{\log_b a-\varepsilon} \sum_{j=0}^{\log_b n-1} (b^\varepsilon)^j\right) = O\left(n^{\log_b a-\varepsilon} \left(\frac{b^{\varepsilon \log_b n}-1}{b^\varepsilon-1}\right)\right) = O\left(n^{\log_b a-\varepsilon} \left(\frac{n^\varepsilon-1}{b^\varepsilon-1}\right)\right)$$

$$\boldsymbol{g(n) = O\left(n^{\log_b a}\right)}$$

Master Method: Proof (6)

- Lemma 2: Proof 2: $f(n) = \theta\left(n^{\log_b a}\right)$;

$$g(n) = \theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right) = \theta\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n^{\log_b a}}{b^{j \log_b a}}\right)\right)$$

$$g(n) = \theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n-1} 1\right) = \theta\left(n^{\log_b a} \log_b n\right)$$

$$\boldsymbol{g(n) = \Theta\left(n^{log_b a} \log n\right)}$$

Master Method: Proof (7)

- Lemma 2: Proof 3: $f(n) = \Omega\left(n^{\log_b a+\varepsilon}\right)$ for $\varepsilon > 0$;

$$g(n) = \Omega\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a+\varepsilon}\right) = \Omega\left(\sum_{j=0}^{\log_b n-1} a^j \left(\frac{n}{b^j}\right)^{\log_b a+\varepsilon}\right)$$

$$g(n) = \Omega\left(n^{\log_b a+\varepsilon} \sum_{j=0}^{\log_b n-1} a^j \left(\frac{1}{b^{\log_b a}{}_b{}^{\varepsilon}}\right)^j\right) = \Omega\left(n^{\log_b a+\varepsilon} \sum_{j=0}^{\log_b n-1} \left(\frac{a}{b^{\log_b a}{}_b{}^{\varepsilon}}\right)^j\right)$$

$$g(n) = \Omega\left(n^{\log_b a+\varepsilon} \sum_{j=0}^{\log_b n-1} \left(\frac{1}{b^{\varepsilon}}\right)^j\right) = \Omega\left(n^{\log_b a+\varepsilon} \left(\frac{b}{b^{\varepsilon}(b-1)}\right)\right) = \Omega\left(n^{\log_b a+\varepsilon}\right)$$

$$\boldsymbol{g(n) = \Omega(f(n))}$$

Master Method: Proof (8)

- Lemma 2: Proof 3: $f(n) = \Omega\left(n^{\log_b a+\varepsilon}\right)$; $af(n/b) \leq cf(n)$; for $\varepsilon > 0, c < 1$

We already proof that $\boldsymbol{g(n) = \Omega\left(f(n)\right)}$

$$f(n) \geq \frac{a}{c} f\left(\frac{n}{b}\right) \geq \frac{a^2}{c^2} f\left(\frac{n}{b^2}\right) \geq \cdots \geq \frac{a^i}{c^i} f\left(\frac{n}{b^i}\right);$$

$$g(n) = \sum_{j=0}^{\log_b n-1} a^j f\left(\frac{n}{b^j}\right) \leq \sum_{j=0}^{\log_b n-1} c^j f(n) + O(1) = f(n) \sum_{j=0}^{\log_b n-1} c^j + O(1)$$

$$g(n) \leq f(n)\left(\frac{1}{1-c}\right) + O(1)$$

$$\boldsymbol{g(n) = O\left(f(n)\right)}$$

$$\boldsymbol{g(n) = \Theta\left(f(n)\right)}$$

Master Method: Proof (9)

- Lemma 3:

Given $T(n) = \begin{cases} \theta(1) & if\ n = 1 \\ aT(n/b) + f(n) & if\ n = b^i \end{cases}$ $a \geq 1, b > 1, f(n) > 0$

1. if $f(n) = O\left(n^{\log_b a - \varepsilon}\right)$ for some constant $\varepsilon > 0$, then $T(n) = \Theta\left(n^{\log_b a}\right)$
2. if $f(n) = \Theta\left(n^{\log_b a}\right)$, then $T(n) = \theta\left(n^{\log_b a} \log n\right)$
3. if $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right)$ for some constant $\varepsilon > 0$, and if $af(n/b) \leq cf(n)$ for some $c < 1$, then $T(n) = \Theta(f(n))$

Master Method: Proof (10)

- Lemma 3:

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right)$$

1. if $f(n) = O\left(n^{\log_b a - \varepsilon}\right); T(n) = \Theta\left(n^{\log_b a}\right) + O\left(n^{log_b a}\right) = \Theta\left(n^{\log_b a}\right)$
2. if $f(n) = \theta\left(n^{\log_b a}\right); T(n) = \Theta\left(n^{\log_b a}\right) + \Theta\left(n^{\log_b a} \log n\right) = \Theta\left(n^{\log_b a} \log n\right)$
3. if $f(n) = \Omega\left(n^{\log_b a + \varepsilon}\right); T(n) = \Theta\left(n^{\log_b a}\right) + \Theta(f(n)) = \Theta(f(n))$

Master Method: Proof (11)

- Extending analysis:

$$T(n) = aT(n/b) + f(n)$$

$$T(n) = \Theta\left(n^{\log_b a}\right) + \sum_{j=0}^{\lfloor \log_b n \rfloor - 1} a^j f(n_j); n_j = \underbrace{\lceil \lceil \lceil n/b \rceil / b \rceil \ldots / b \rceil}_{j\ times}$$

All three lemmas can be proofed using similar way but for the above formula.

Exersises: Solve Recurrences

1. $T(n) = T(n-1) + 2$
2. $T(n) = 3T(n-1) + n/2$
3. $T(n) = 3T(n-1) + n/3$
4. $T(n) = 4T(n-1) - 3T(n-2)$
5. $T(n) = 2T(n/2) + n$
6. $T(n) = 3T(n/2) + n$
7. $T(n) = 2T(n/2) + n^2$
8. $T(n) = 2T(n-1) + n^2$
9. $T(n) = T(n/2) + \log n$
10. $T(n) = 2T(n/2) + \sqrt{n}$
11. $T(n) = 2T(n/2) + n \log n$
12. $T(n) = 5T(n/3) + n^2$
13. $T(n) = 2T(n-2) + T(n/4) + n$
14. $T(n) = T(n/3) + n^3$
15. $T(n) = T(n-2) + n^2$
16. $T(n) = T(7n/10) + n$
17. $T(n) = 5T(n/5) + \log n$
18. $T(n) = T(n/2) + T(n/4) + n$
19. $T(n) = 2T(\sqrt{n}) + n$
20. $T(n) = T(n-1) + T(n-2) + 1$

Thank You